

Persist data with Room

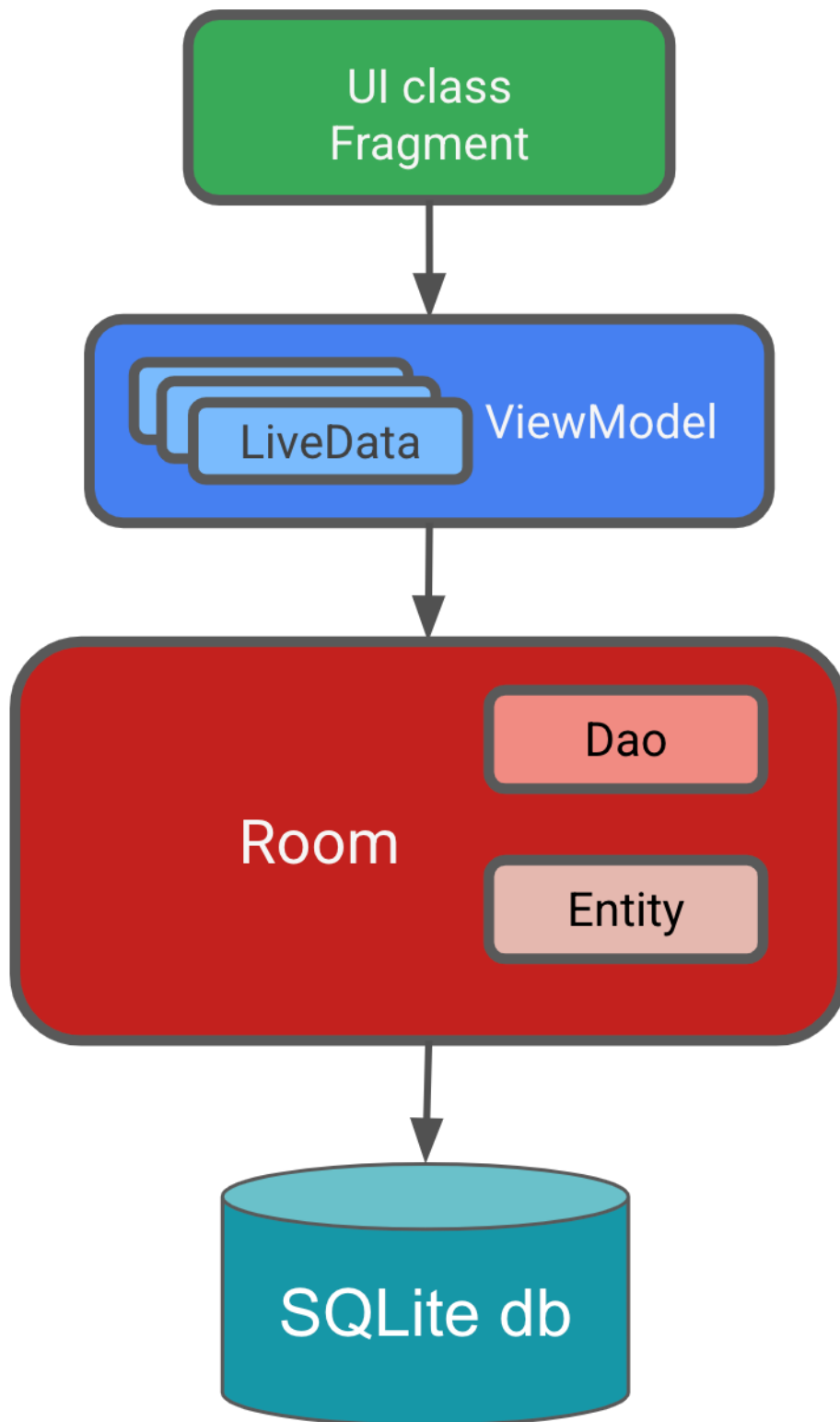
1. Before you begin
2. App overview
3. Starter app overview
4. Main components of Room
5. Create an item Entity
6. Create the item DAO
7. Create a Database instance
8. Add a ViewModel
9. Update AddItemFragment
10. Solution code
11. Summary
12. Learn more

1. Before you begin

Most production quality apps have data that needs to be saved, even after the user closes the app. For example, the app might store a playlist of songs, items on a to-do list, records of expenses and income, a catalog of constellations, or a history of personal data. For most of these cases, you use a database to store this persistent data.

[Room](#) is a persistence library that's part of Android [Jetpack](#). Room is an abstraction layer on top of a [SQLite](#) database. SQLite uses a specialized language (SQL) to perform database operations. Instead of using SQLite directly, Room simplifies the chores of setting up, configuring, and interacting with the database. Room also provides compile-time checks of SQLite statements.

The image below shows how Room fits in with the overall architecture recommended in this course.



Prerequisites

- You know how to build a basic user interface (UI) for an Android app.
- You know how to use activities, fragments, and views.
- You know how to navigate between fragments, using Safe Args to pass data between fragments.
- You are familiar with the Android architecture components `ViewModel`, `LiveData`, and `Flow`, and know how to use `ViewModelProvider.Factory` to instantiate the `ViewModels`.
- You are familiar with concurrency fundamentals.
- You know how to use coroutines for long-running tasks.
- You have a basic understanding of SQL databases and the SQLite language.

What you'll learn

- How to create and interact with the SQLite database using the Room library.
- How to create an entity, DAO, and database classes.
- How to use a data access object (DAO) to map Kotlin functions to SQL queries.

What you'll build

- You'll build an Inventory app that saves inventory items into the SQLite database.

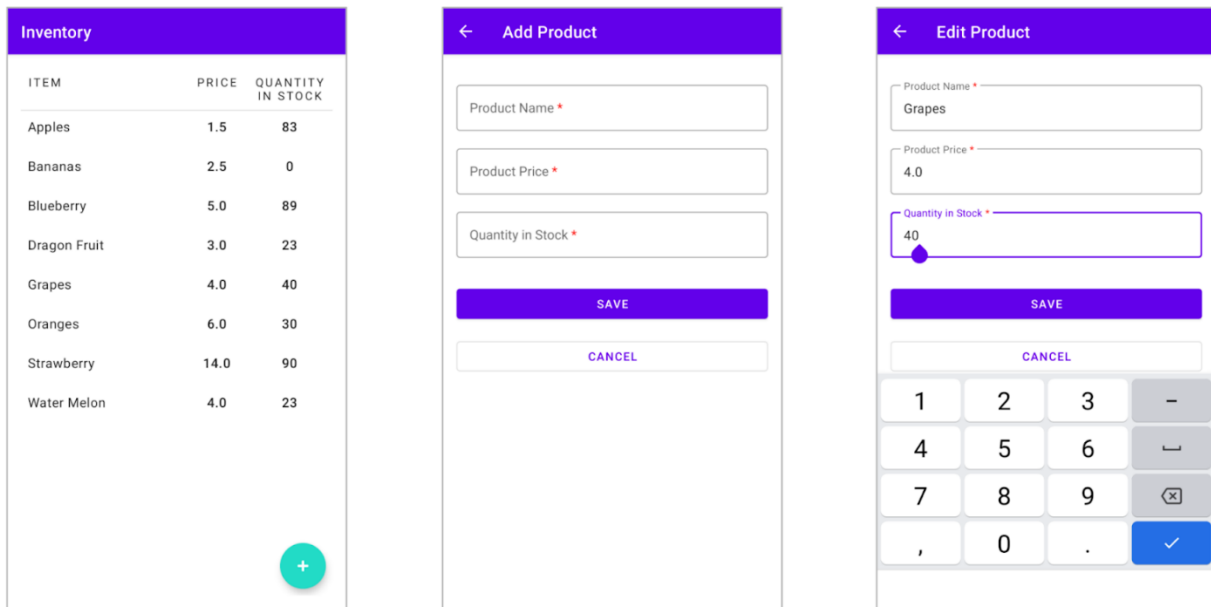
What you need

- Starter code for the **Inventory** app.
- A computer with Android Studio installed.

2. App overview

In this codelab, you will work with a starter app called Inventory app, and add the database layer to it using the Room library. The final version of the app displays a list items from the inventory database using a `RecyclerView`. The user will have options to add a new item, update an existing item, and delete an item from the inventory database (you'll complete the app's functionality in the next codelab).

Below are screenshots from the final version of the app.



Note: The above screenshots are from the final version of the app at the end of the pathway, not at the end of this codelab. These screenshots are included here to give you an idea of the final version of the app.

3. Starter app overview

Download the starter code for this codelab

This codelab provides starter code for you to extend with features taught in this codelab. Starter code may contain code that is familiar to you from previous codelabs, and also code that is unfamiliar to you that you will learn about in later codelabs.

If you use the starter code from GitHub, note that the folder name is `android-basics-kotlin-inventory-app-starter`. Select this folder when you open the project in Android Studio.

Starter Code URL:

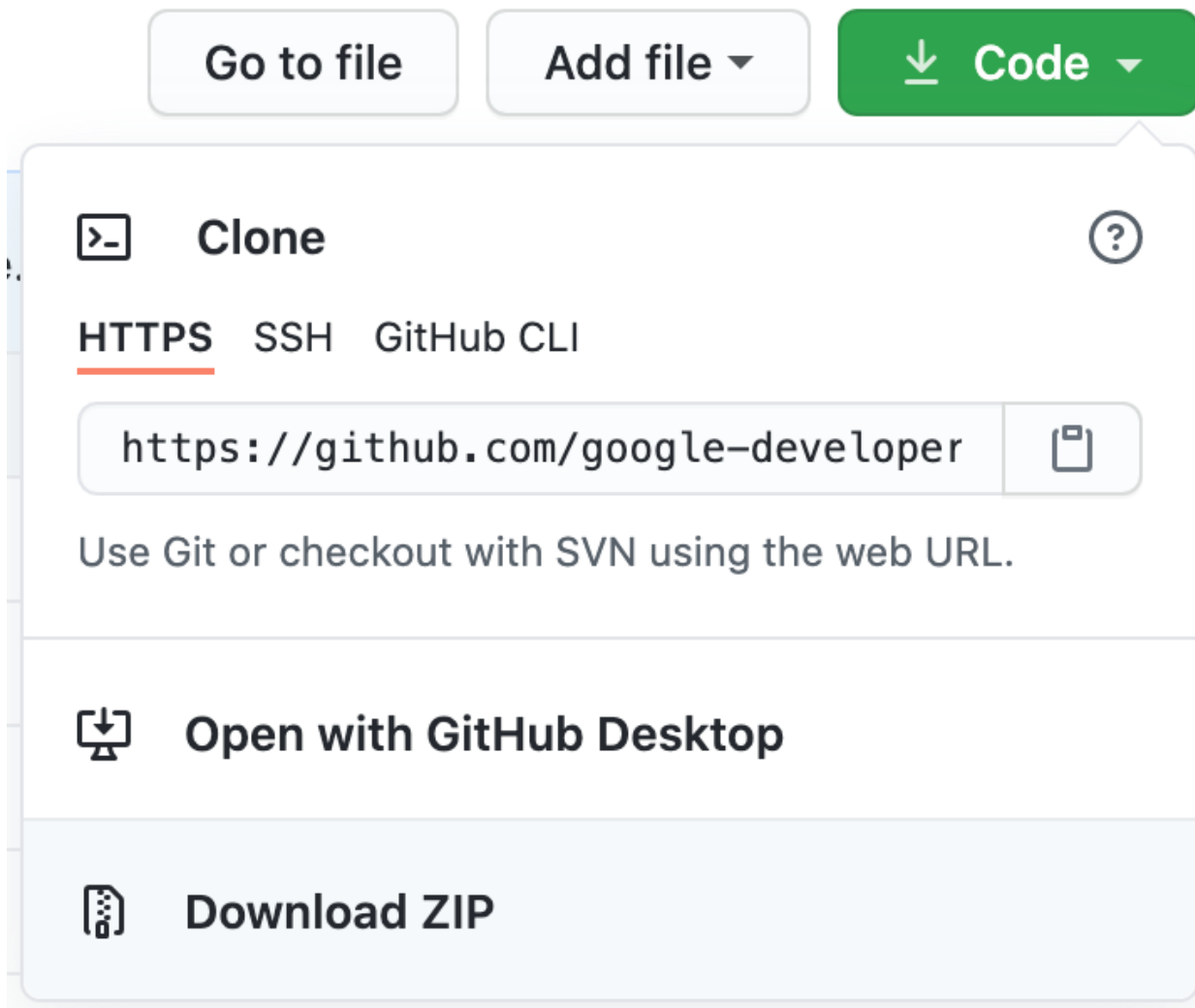
<https://github.com/google-developer-training/android-basics-kotlin-inventory-app/tree/starter>

Branch name with starter code: `starter`

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

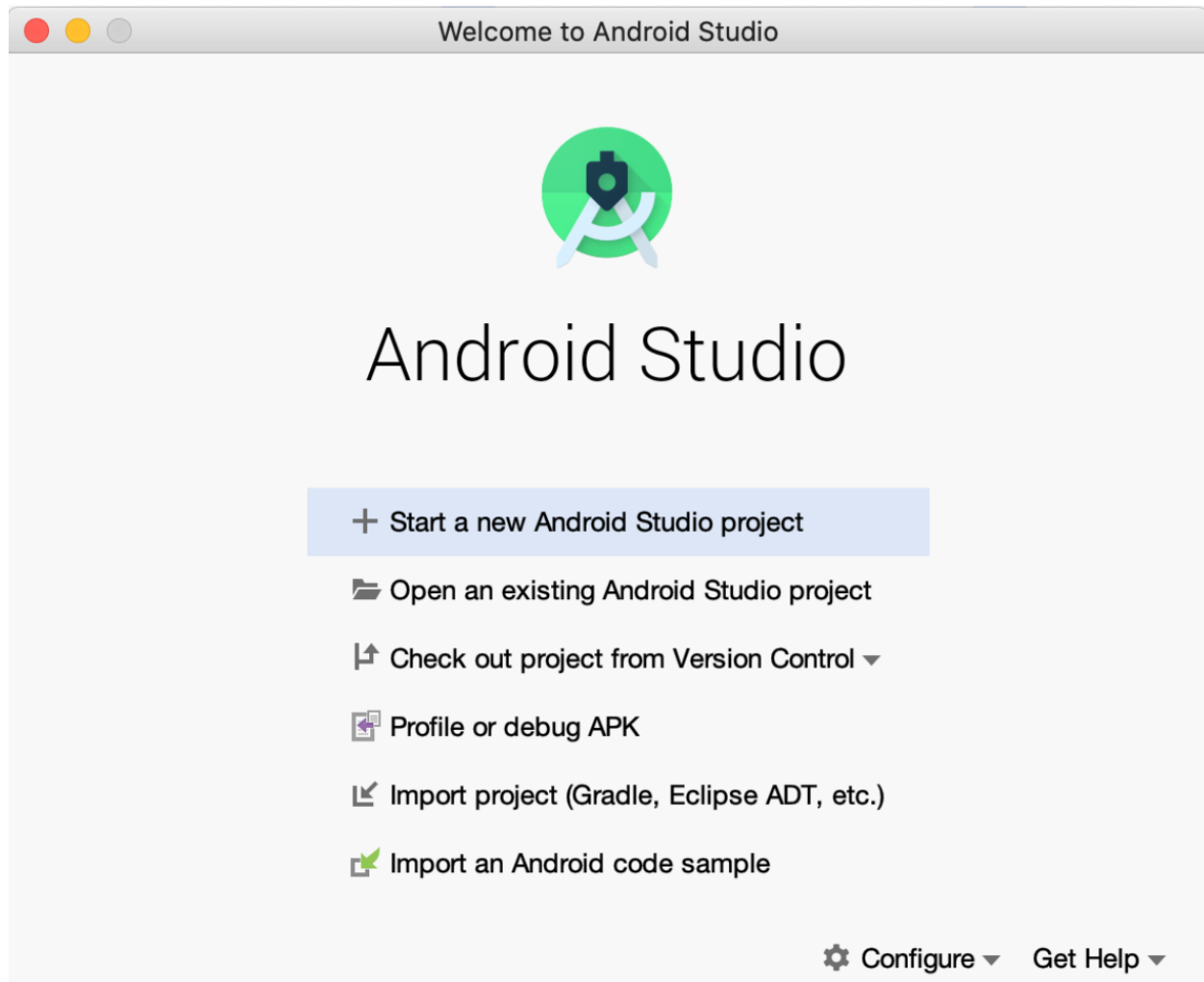
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



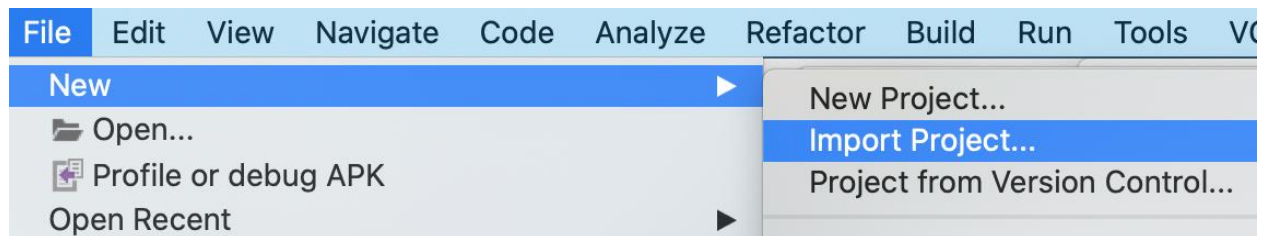
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.

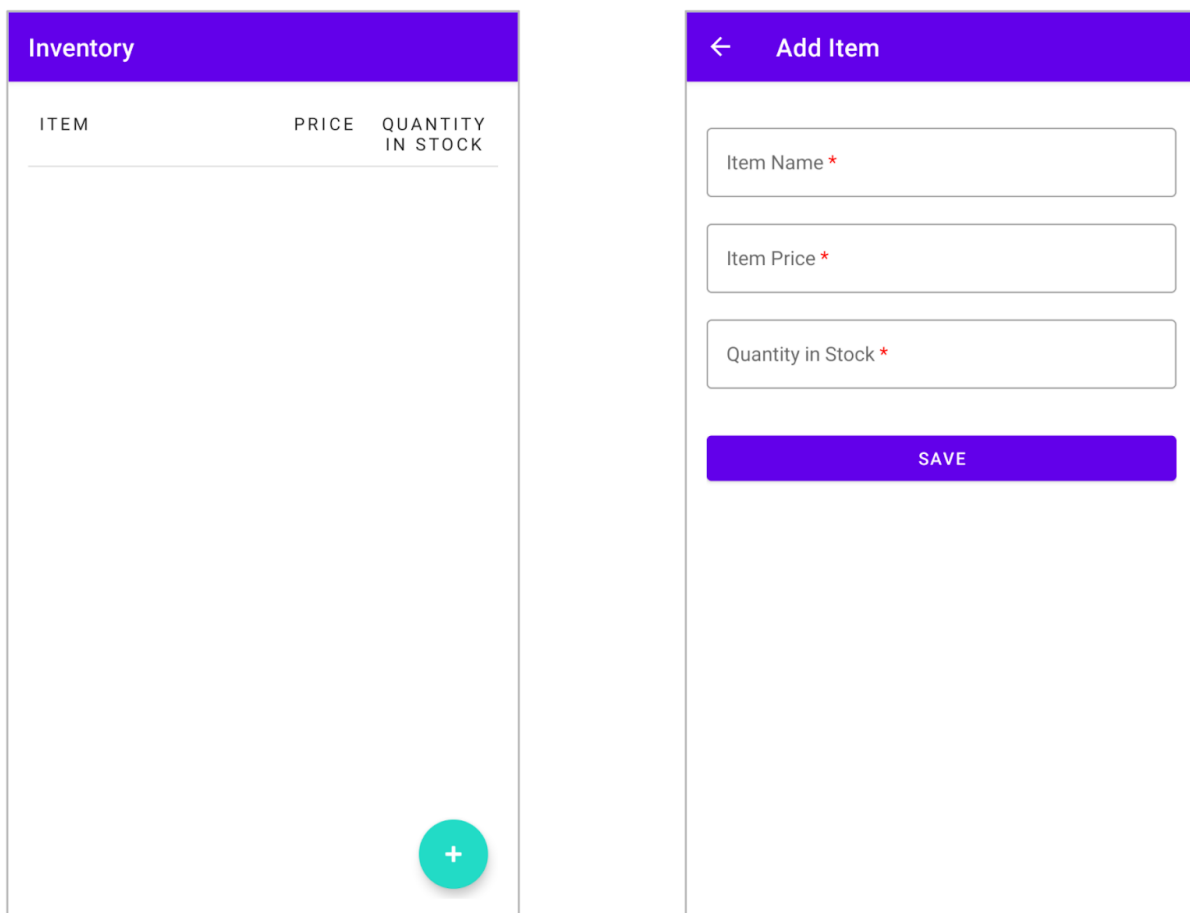


6. Click the **Run** button to build and run the app. Make sure it builds as expected.

7. Browse the project files in the **Project** tool window to see how the app is set-up.

Starter code overview

1. Open the project with the starter code in Android Studio.
2. Run the app on an Android device, or on an emulator. Make sure the emulator or connected device is running API level 26 or higher. [Database Inspector](#) works best on emulator/devices running API level 26.
3. The app shows no inventory data. Notice the FAB to add new items to the database.
4. Click on the FAB. The app navigates to a new screen where you can enter details for the new item.



Problems with the starter code

1. In the **Add Item** screen enter an item's details. Tap **Save**. The add item fragment is not closed. Navigate back using the system back key. The new item is not saved and is not listed on the inventory screen. Notice that the app is incomplete and the **Save** button functionality is not implemented.

←

Add Item

Item Name *

Apples

Item Price *

\$2.00

Quantity in Stock *

100

SAVE

1

2

3

-

4

5

6

_

7

8

9

⌫

,

0

.

✓

Inventory		
ITEM	PRICE	QUANTITY IN STOCK

In this codelab, you will add the database portion of an app that saves the inventory details in the SQLite database. You will be using the Room persistence library to interact with the SQLite database.

Code walkthrough

The starter code you downloaded has the screen layouts pre-designed for you. In this pathway, you will focus on implementing the database logic. Here is a brief walkthrough of some of the files to get you started.

main_activity.xml

The main activity that hosts all the other fragments in the app. The `onCreate()` method retrieves `NavController` from the `NavHostFragment` and sets up the action bar for use with the `NavController`.

item_list_fragment.xml

The first screen shown in the app. It mainly contains a RecyclerView and a FAB. You will implement the RecyclerView later in the pathway.

fragment_add_item.xml

This layout contains text fields for entering the details of the new inventory item to be added.

ItemListFragment.kt

This fragment contains mostly boilerplate code. In the `onViewCreated()` method, click listener is set on FAB to navigate to the add item fragment.

AddItemFragment.kt

This fragment is used to add new items into the database. The `onCreateView()` function initializes the binding variable and the `onDestroyView()` function hides the keyboard before destroying the fragment.

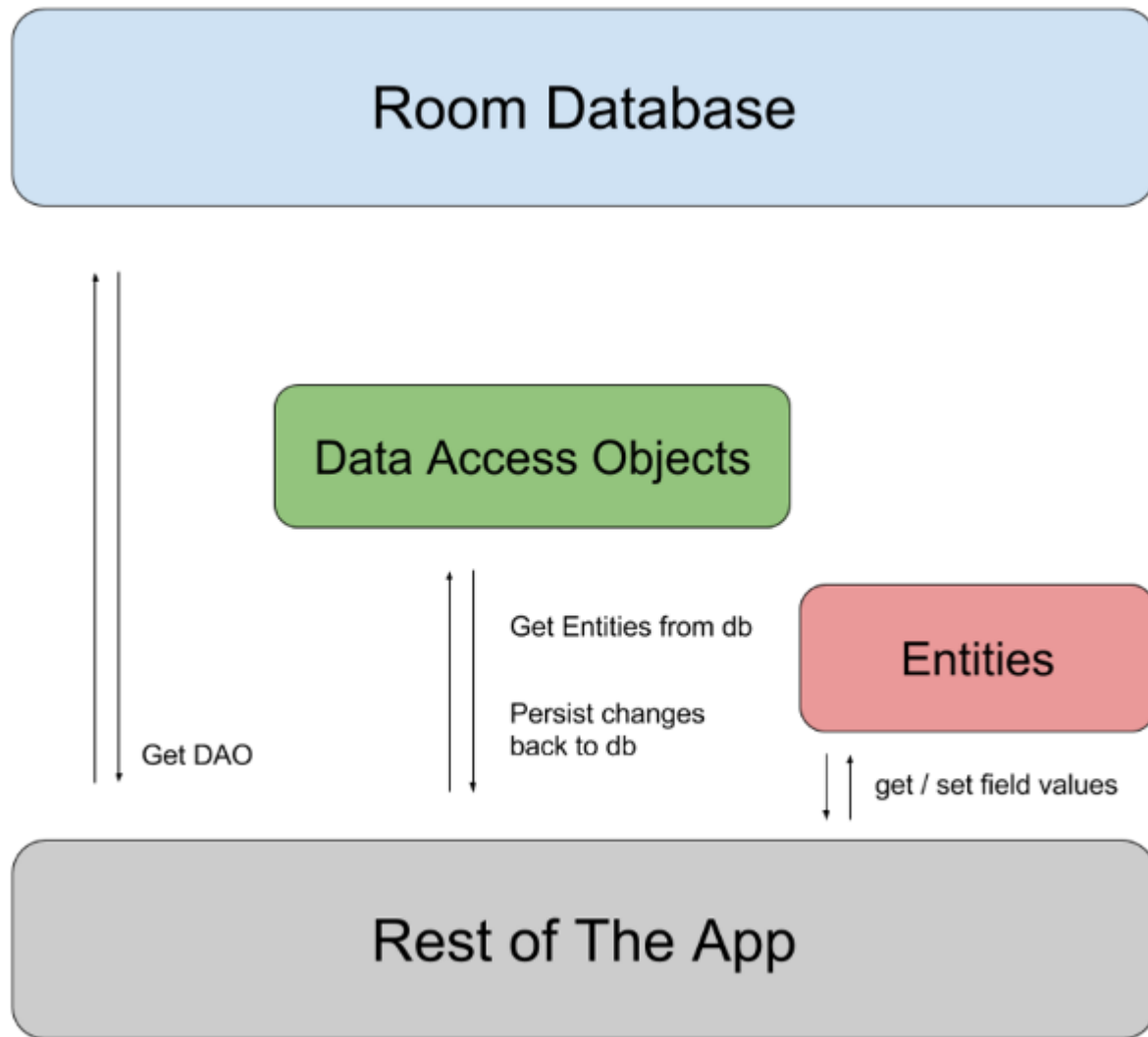
4. Main components of Room

Kotlin provides an easy way to deal with data by introducing data classes. This data is accessed and possibly modified using function calls. However, in the database world, you need *tables* and *queries* to access and modify data. The following components of [Room](#) make these workflows seamless.

There are three major components in Room:

- [Data entities](#) represent tables in your app's database. They are used to update the data stored in rows in tables, and to create new rows for insertion.
- [Data access objects \(DAOs\)](#) provide methods that your app uses to retrieve, update, insert, and delete data in the database.
- [Database class](#) holds the database and is the main access point for the underlying connection to your app's database. The database class provides your app with instances of the DAOs associated with that database.

You will implement and learn more about these components later in the codelab. The following diagram demonstrates how the components of the Room work together to interact with the database.



Add Room libraries

In this task, you'll add the required Room component libraries to your Gradle files.

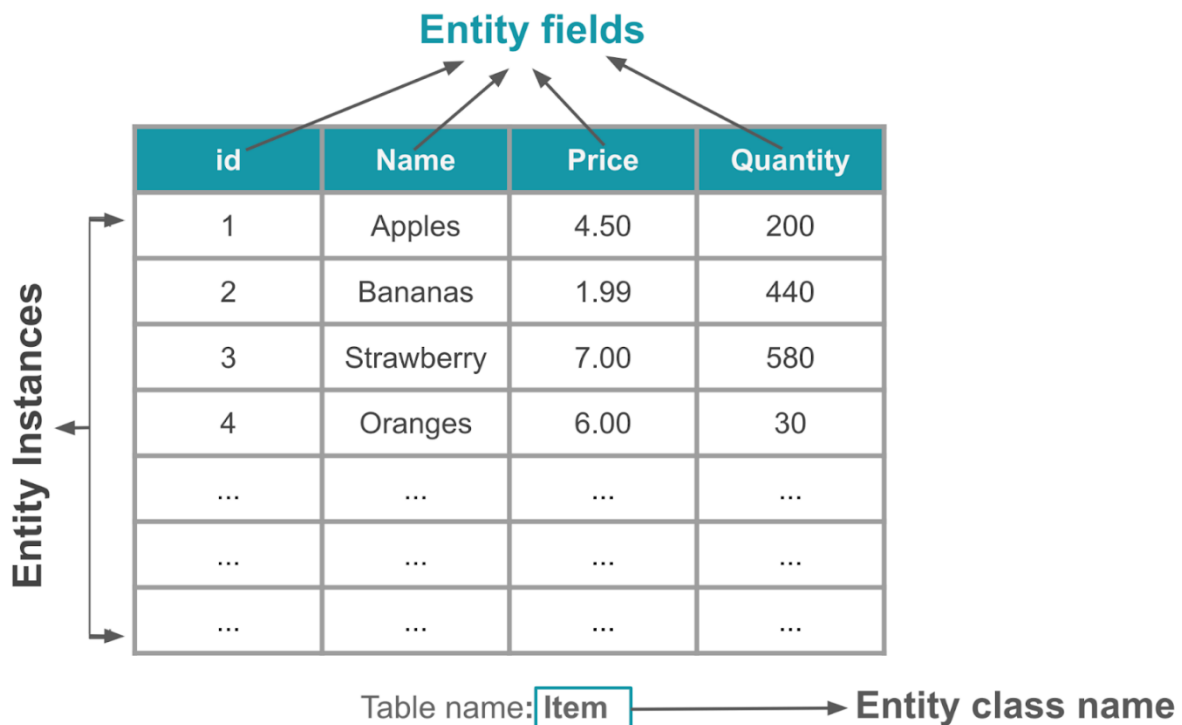
1. Open module level gradle file, `build.gradle` (Module: `InventoryApp.app`) In the `dependencies` block, add the following dependencies for the Room library.

```
// Room
implementation "androidx.room:room-runtime:$room_version"
kapt "androidx.room:room-compiler:$room_version"
implementation "androidx.room:room-ktx:$room_version"
```

Note: For the library dependencies in your gradle file, always use the most current stable release version numbers from the [AndroidX releases](#) page.

5. Create an item Entity

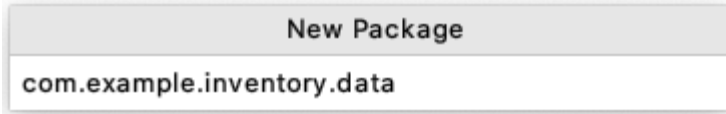
[Entity](#) class defines a table, and each instance of this class represents a row in the database table. The entity class has mappings to tell Room how it intends to present and interact with the information in the database. In your app, the entity is going to hold information about inventory items such as item name, item price and stock available.



`@Entity` annotation marks a class as a database Entity class. For each Entity class a database table is created to hold the items. Each field of the Entity is represented as a column in the database, unless it is denoted otherwise (see [Entity](#) docs for details). Every entity instance that is stored in the database must have a primary key. The [primary key](#) is used to uniquely identify every record/entry in your database tables. Once assigned, the primary key cannot be modified, it represents the entity object as long as it exists in the database.

In this task, you will create an Entity class. Define fields to store the following inventory information for each item.

- An `Int` to store the primary key.
 - A `String` to store the item name.
 - A `double` to store the item price.
 - An `Int` to store the quantity in stock.
1. Open starter code in the Android Studio.
 2. Create a package called `data` under `com.example.inventory` base package.



3. Inside the `data` package, create a Kotlin class called `Item`. This class will represent a database entity in your app. In the next step you will add corresponding fields to store inventory information.
4. Update the `Item` class definition with the following code. Declare `id` of type `Int`, `itemName` of type `String`, `itemPrice` of type `Double`, and `quantityInStock` of type `Int` as parameters for the primary constructor. Assign a default value of 0 to `id`. This will be the primary key, an ID to uniquely identify every record/entry in your `Item` table.

```
class Item(  
    val id: Int = 0,  
    val itemName: String,  
    val itemPrice: Double,  
    val quantityInStock: Int  
)
```

Refresher on primary constructor: The primary constructor is part of the class header in a Kotlin class: it goes after the class name (and optional type parameters).

Data classes

Data classes are primarily used to hold data in Kotlin. They are marked with the keyword `data`. Kotlin data class objects have some extra benefits, the compiler automatically generates utilities for comparing, printing and copying such as `toString()`, [copy\(\)](#), and `equals()`.

Example:

```
// Example data class with 2 properties.  
data class User(val first_name: String, val last_name: String){  
}
```

To ensure consistency and meaningful behavior of the generated code, data classes have to fulfill the following requirements:

- The primary constructor needs to have at least one parameter.
- All primary constructor parameters need to be marked as `val` or `var`.
- Data classes cannot be `abstract`, `open`, `sealed` or `inner`.

Warning: The compiler only uses the properties defined inside the primary constructor for the automatically generated functions. The properties declared inside the class body are excluded from the generated implementations.

To learn more about Data classes, check out the [documentation](#).

5. Convert the `Item` class to a data class by prefixing its class definition with `data` keyword.

```
data class Item(  
    val id: Int = 0,  
    val itemName: String,  
    val itemPrice: Double,  
    val quantityInStock: Int  
)
```

6. Above the `Item` class declaration, annotate the data class with `@Entity`. Use `tableName` argument to give the `item` as the SQLite table name.

```
@Entity(tableName = "item")  
data class Item(  
    ...  
)
```

Important: When prompted by Android Studio, import `Entity` and all other Room annotations (which you will use later in the codelab) from the `androidx` library. For example, `androidx.room.Entity`.

Note: `@Entity` annotation has several possible arguments. By default (no arguments to `@Entity`), the table name will be the same as the class. The `tableName` argument lets you give a different or a more helpful table name. This argument for the `tableName` is optional, but highly recommended. For simplicity you will give the same name as the class name, that is `item`. There are several other arguments for `@Entity` you can investigate in the [documentation](#).

7. To identify the `id` as the primary key, annotate the `id` property with `@PrimaryKey`. Set the parameter `autoGenerate` to `true` so that Room generates the ID for each entity. This guarantees that the ID for each item is unique.

```
@Entity(tableName = "item")  
data class Item(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int = 0,  
    ...  
)
```

8. Annotate the remaining properties with `@ColumnInfo`. The `ColumnInfo` annotation is used to customise the column associated with the particular field. For example, when using the `name` argument, you can specify a different column name for the field rather than the variable name. Customize the property names using parameters as shown below. This approach is similar to using `tableName` to specify a different name to the database.

```
import androidx.room.ColumnInfo  
import androidx.room.Entity  
import androidx.room.PrimaryKey  
  
@Entity  
data class Item(  
    ...  
)
```

```

@PrimaryKey(autoGenerate = true)
val id: Int = 0,
@ColumnInfo(name = "name")
val itemName: String,
@ColumnInfo(name = "price")
val itemPrice: Double,
@ColumnInfo(name = "quantity")
val quantityInStock: Int
)

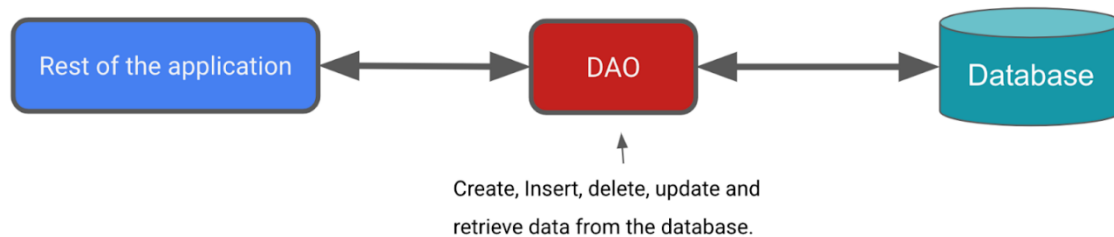
```

6. Create the item DAO

Data Access Object (DAO)

The Data Access Object (DAO) is a pattern used to separate the persistence layer with the rest of the application by providing an abstract interface. This isolation follows the [single responsibility principle](#), which you have seen in the previous code labs.

The functionality of the DAO is to hide all the complexities involved in performing the database operations in the underlying persistence layer from the rest of the application. This allows the data access layer to be changed independently of the code that uses the data.



In this task, you define a [Data Access Object](#) (DAO) for the Room. Data access objects are the main components of Room that are responsible for defining the interface that accesses the database.

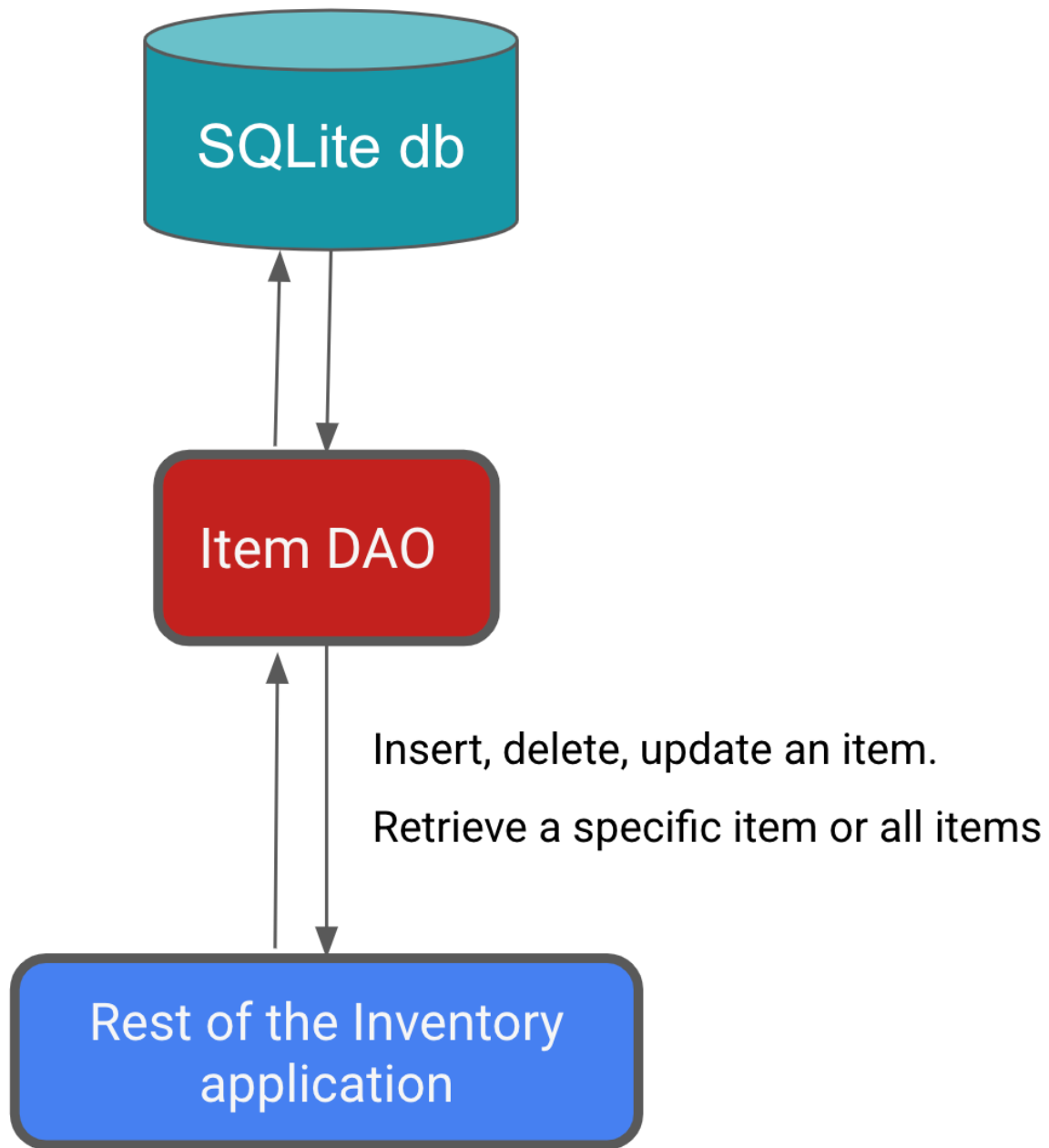
The DAO you will create will be a custom interface providing convenience methods for querying/retrieving, inserting, deleting, and updating the database. Room will generate an implementation of this class at compile time.

For common database operations, the `Room` library provides convenience annotations, such as `@Insert`, `@Delete`, and `@Update`. For everything else, there is the `@Query` annotation. You can write any query that's supported by SQLite.

As an added bonus, as you write your queries in Android Studio, the compiler checks your SQL queries for syntax errors.

For the inventory app, you need to be able to do the following:

- **Insert** or add a new item.
- **Update** an existing item to update name, price, and quantity.
- **Get** a specific item based on its primary key, `id`.
- **Get all items**, so you can display them.
- **Delete** an entry in the database.



Now, implement the item DAO in your app:

1. In the `data` package, create Kotlin class `ItemDao.kt`.
2. Change the class definition to `interface` and annotate with `@Dao`.

```
@Dao
interface ItemDao {
}
```

3. Inside the body of the interface, add an `@Insert` annotation. Below the `@Insert`, add an `insert()` function that takes an instance of the `Entity` class `item` as its argument. The database operations can take a long time to execute, so they should run on a separate thread. Make the function a suspend function, so that this function can be called from a coroutine.

```
@Insert
suspend fun insert(item: Item)
```

4. Add an argument `OnConflict` and assign it a value of `OnConflictStrategy.IGNORE`. The argument `OnConflict` tells the Room what to do in case of a conflict. The `OnConflictStrategy.IGNORE` strategy ignores a new item if its primary key is already in the database. To know more about the available conflict strategies, check out the [documentation](#).

```
@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insert(item: Item)
```

Now the Room will generate all the necessary code to insert the `item` into the database. When you call `insert()` from your Kotlin code, Room executes a SQL query to insert the entity into the database. (Note: The function can be named anything you want; it doesn't have to be called `insert()`.)

5. Add an `@Update` annotation with an `update()` function for one `item`. The entity that's updated has the same key as the entity that's passed in. You can update some or all of the entity's other properties. Similar to the `insert()` method, make the following `update()` method `suspend`.

```
@Update
suspend fun update(item: Item)
```

6. Add `@Delete` annotation with a `delete()` function to delete item(s). Make it a suspend method. The `@Delete` annotation deletes one item, or a list of items. (Note: You need to pass the entity(s) to be deleted, if you don't have the entity you may have to fetch it before calling the `delete()` function.)

```
@Delete
suspend fun delete(item: Item)
```

There is no convenience annotation for the remaining functionality, so you have to use the `@Query` annotation and supply SQLite queries.

7. Write a SQLite query to retrieve a particular item from the `item` table based on the given `id`. You will then add Room annotation and use a modified version of the following query in the later steps. In next steps, you will also change this into a DAO method using Room.
8. Select all columns from the `item`
9. `WHERE` the `id` matches a specific value.

Example:

```
SELECT * from item WHERE id = 1
```

8. Change the above SQL query to use with the Room annotation and an argument. Add a `@Query` annotation, supply the query as a string parameter to the `@Query` annotation. Add a `String` parameter to `@Query` that is a SQLite query to retrieve an item from the `item` table.
9. Select all columns from the `item`
10. `WHERE` the `id` matches the `:id` argument. Notice the `:id`. You use the colon notation in the query to reference arguments in the function.

```
@Query("SELECT * from item WHERE id = :id")
```

9. Below the `@Query` annotation add `getItem()` function that takes an `Int` argument and returns a `Flow<Item>`.

```
@Query("SELECT * from item WHERE id = :id")  
fun getItem(id: Int): Flow<Item>
```

Using `Flow` or `LiveData` as return type will ensure you get notified whenever the data in the database changes. It is recommended to use `Flow` in the persistence layer. The Room keeps this `Flow` updated for you, which means you only need to explicitly get the data once. This is helpful to update the inventory list, which you will implement in the next codelab. Because of the `Flow` return type, Room also runs the query on the background thread. You don't need to explicitly make it a `suspend` function and call inside a coroutine scope.

You may need to import `Flow` from `kotlinx.coroutines.flow.Flow`.

10. Add a `@Query` with a `getItems()` function:
11. Have the SQLite query return all columns from the `item` table, ordered in ascending order.
12. Have `getItems()` return a list of `Item` entities as `Flow`. Room keeps this `Flow` updated for you, which means you only need to explicitly get the data once.

```
@Query("SELECT * from item ORDER BY name ASC")  
fun getItems(): Flow<List<Item>>
```

11. Though you won't see any visible changes, run your app to make sure it has no errors.

7. Create a Database instance

In this task, you create a [RoomDatabase](#) that uses the `Entity` and `DAO` that you created in the previous task. The database class defines the list of entities and data access objects. It is also the main access point for the underlying connection.

The [Database](#) class provides your app with instances of the `DAOs` you've defined. In turn, the app can use the `DAOs` to retrieve data from the database as instances of the associated data entity objects. The app can also use the defined data entities to update rows from the corresponding tables, or to create new rows for insertion.

You need to create an abstract `RoomDatabase` class, annotated with `@Database`. This class has one method that either creates an instance of the `RoomDatabase` if it doesn't exist, or returns the existing instance of the `RoomDatabase`.

Here's the general process for getting the `RoomDatabase` instance:

- Create a `public abstract` class that extends `RoomDatabase`. The new abstract class you defined acts as a database holder. The class you defined is abstract, because `Room` creates the implementation for you.
- Annotate the class with `@Database`. In the arguments, list the entities for the database and set the version number.
- Define an abstract method or property that returns an `ItemDao` Instance and the `Room` will generate the implementation for you.
- You only need one instance of the `RoomDatabase` for the whole app, so make the `RoomDatabase` a singleton.
- Use `Room`'s [Room.databaseBuilder](#) to create your (`item_database`) database only if it doesn't exist. Otherwise, return the existing database.

Tip: The following code can be used as a template for your future projects. The way you create the `RoomDatabase` instance is similar to the process defined above. You may have to replace the entities and `Dao`'s specific to your app.

Create the Database

1. In the `data` package, create a Kotlin class `ItemRoomDatabase.kt`.
2. In the `ItemRoomDatabase.kt` file, make `ItemRoomDatabase` class as an abstract class that extends `RoomDatabase`. Annotate the class with `@Database`. You will fix the missing parameters error in the next step.

```
@Database
abstract class ItemRoomDatabase : RoomDatabase() {}
```

3. The `@Database` annotation requires several arguments, so that `Room` can build the database.
 - Specify the `Item` as the only class with the list of `entities`.
 - Set the `version` as 1. Whenever you change the schema of the database table, you'll have to increase the version number.
 - Set `exportSchema` to `false`, so as not to keep schema version history backups.

```
@Database(entities = [Item::class], version = 1, exportSchema = false)
```

4. The database needs to know about the DAO. Inside the body of the class, declare an abstract function that returns the `ItemDao`. You can have multiple DAOs.

```
abstract fun itemDao(): ItemDao
```

5. Below the abstract function, define a `companion` object. The companion object allows access to the methods for creating or getting the database using the class name as the qualifier.

```
companion object {}
```

6. Inside the `companion` object, declare a private nullable variable `INSTANCE` for the database and initialize it to `null`. The `INSTANCE` variable will keep a reference to the database, when one has been created. This helps in maintaining a single instance of the database opened at a given time, which is an expensive resource to create and maintain.

Annotate `INSTANCE` with `@Volatile`. The value of a volatile variable will never be cached, and all writes and reads will be done to and from the main memory. This helps make sure the value of `INSTANCE` is always up-to-date and the same for all execution threads. It means that changes made by one thread to `INSTANCE` are visible to all other threads immediately.

```
@Volatile
private var INSTANCE: ItemRoomDatabase? = null
```

7. Below `INSTANCE`, while still inside the `companion` object, define a `getDatabase()` method with a `Context` parameter that the database builder will need. Return a type `ItemRoomDatabase`. You'll see an error because `getDatabase()` isn't returning anything yet.

```
fun getDatabase(context: Context): ItemRoomDatabase {}
```

8. Multiple threads can potentially run into a race condition and ask for a database instance at the same time, resulting in two databases instead of one. Wrapping the code to get the database inside a `synchronized` block means that only one thread of execution at a time can enter this block of code, which makes sure the database only gets initialized once.

Inside `getDatabase()`, return `INSTANCE` variable or if `INSTANCE` is null, initialize it inside a `synchronized{}` block. Use the elvis operator(`?:`) to do this. Pass in `this` the companion object, that you want to be locked inside the function block. You will fix the error in the later steps.

```
return INSTANCE ?: synchronized(this) { }
```

9. Inside the `synchronized` block, create a `val` instance variable, and use the database builder to get the database. You will still have errors which you will fix in the next steps.

```
val instance = Room.databaseBuilder()
```

10. At the end of the `synchronized` block, return `instance`.

```
return instance
```

11. Inside the `synchronized` block, initialize the `instance` variable, and use the database builder to get a database. Pass in the application context, the database class, and a name for the database, `item_database` to the `Room.databaseBuilder()`.

```
val instance = Room.databaseBuilder(  
    context.applicationContext,  
    ItemRoomDatabase::class.java,  
    "item_database"  
)
```

Android Studio will generate a Type Mismatch error. To remove this error, you'll have to add a migration strategy and `build()` in the following steps.

12. Add the required migration strategy to the builder. Use `.fallbackToDestructiveMigration()`.

Normally, you would have to provide a migration object with a migration strategy for when the schema changes. A *migration object* is an object that defines how you take all rows with the old schema and convert them to rows in the new schema, so that no data is lost. [Migration](#) is beyond the scope of this codelab. A simple solution is to destroy and rebuild the database, which means that the data is lost.

```
.fallbackToDestructiveMigration()
```

13. To create the database instance, call `.build()`. This should remove the Android Studio errors.

```
.build()
```

14. Inside the `synchronized` block, assign `INSTANCE = instance`.

```
INSTANCE = instance
```

15. At the end of the `synchronized` block, return `instance`. Your final code should look like this:

```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Item::class], version = 1, exportSchema = false)
abstract class ItemRoomDatabase : RoomDatabase() {

    abstract fun itemDao(): ItemDao

    companion object {
        @Volatile
        private var INSTANCE: ItemRoomDatabase? = null
        fun getDatabase(context: Context): ItemRoomDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    ItemRoomDatabase::class.java,
                    "item_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
                return instance
            }
        }
    }
}
```

16. Build your code to make sure there are no errors.

Implement Application class

In this task you will instantiate the database instance in the Application class.

1. Open `InventoryApplication.kt`, create a `val` called `database` of the type `ItemRoomDatabase`. Instantiate the database instance by calling `getDatabase()` on `ItemRoomDatabase` passing in the context. Use `lazy delegate` so the instance database is lazily created when you first need/access the reference (rather than when the app starts). This will create the database (the physical database on the disk) on the first access.

```
import android.app.Application
import com.example.inventory.data.ItemRoomDatabase

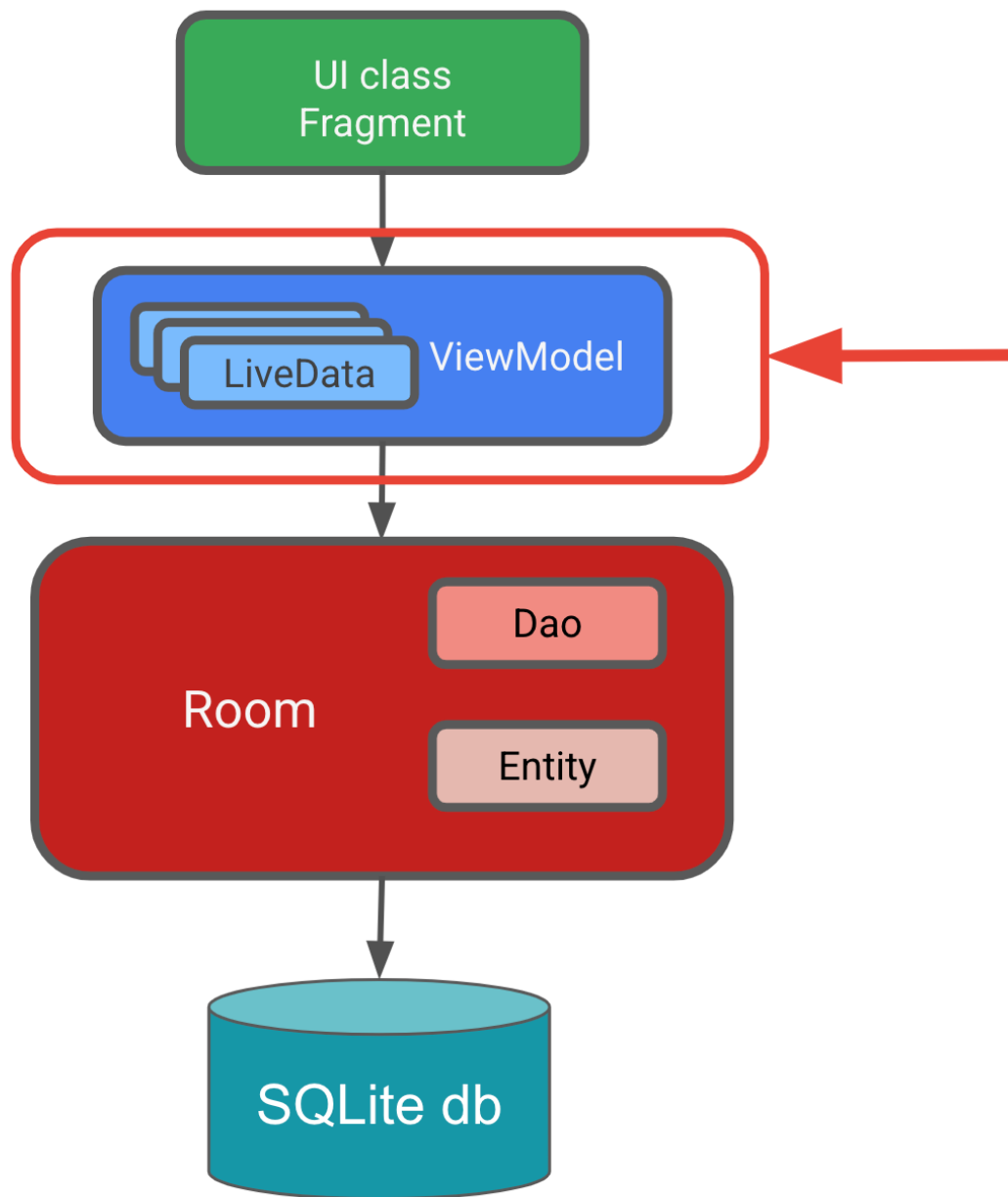
class InventoryApplication : Application() {
    val database: ItemRoomDatabase by lazy {
        ItemRoomDatabase.getDatabase(this)
    }
}
```

You will use this `database` instance later in the codelab when creating a `ViewModel` instance.

You now have all the building blocks for working with your Room. This code compiles and runs, but you have no way of telling if it actually works. So, this is a good time to add a new item to your Inventory database to test your database. To accomplish this, you need a `ViewModel` to talk to the database.

8. Add a ViewModel

You have thus far created a database and the UI classes were part of the starter code. To save the app's transient data and to also access the database, you need a `ViewModel`. Your Inventory `ViewModel` will interact with the database via the DAO, and provide data to the UI. All database operations will have to be run away from the main UI thread, you'll do that using coroutines and [`viewModelScope`](#).



Create Inventory ViewModel

1. In the `com.example.inventory` package, create a Kotlin class file `InventoryViewModel.kt`.
2. Extend the `InventoryViewModel` class from the `ViewModel` class. Pass in the `ItemDao` object as a parameter to the default constructor.

```
class InventoryViewModel(private val itemDao: ItemDao) : ViewModel() {}
```

3. At the end of the `InventoryViewModel.kt` file outside the class, add `InventoryViewModelFactory` class to instantiate the `InventoryViewModel` instance. Pass in the same constructor parameter as the `InventoryViewModel` that is the `ItemDao` instance. Extend the class from the `ViewModelProvider.Factory` class. You will fix the error regarding the unimplemented methods in the next step.

```
class InventoryViewModelFactory(private val itemDao: ItemDao) :  
    ViewModelProvider.Factory {  
}
```

4. Click on the red bulb and select **Implement Members**, or you can override the `create()` method inside the `ViewModelProvider.Factory` class as follows, which takes any class type as an argument and returns a `ViewModel` object.

```
override fun <T : ViewModel?> create(modelClass: Class<T>): T {  
    TODO("Not yet implemented")  
}
```

5. Implement the `create()` method. Check if the `modelClass` is the same as the `InventoryViewModel` class and return an instance of it. Otherwise, throw an exception.

```
if (modelClass.isAssignableFrom(InventoryViewModel::class.java)) {  
    @Suppress("UNCHECKED_CAST")  
    return InventoryViewModel(itemDao) as T  
}  
throw IllegalArgumentException("Unknown ViewModel class")
```

Tip: The creation of the `ViewModel` factory is mostly boilerplate code, so you can reuse this code for future `ViewModel` factories.

Populate the ViewModel

In this task, you will populate the `InventoryViewModel` class to add inventory data to the database. Observe the `Item` entity and **Add Item** screen in the Inventory app.

```
@Entity  
data class Item(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int = 0,  
    @ColumnInfo(name = "name")  
    val itemName: String,  
    @ColumnInfo(name = "price")  
    val itemPrice: Double,  
    @ColumnInfo(name = "quantity")  
    val quantityInStock: Int  
)
```

← Add Item

Item Name *

Apples

Item Price *

\$ 2.00

Quantity in Stock *

100

SAVE

1

2

3

-

4

5

6

⌋

7

8

9

⌫

,

0

.

✓

You need the name, price, and stock in hand for that particular item in order to add an entity to the database. Later in the codelab, you will use the **Add Item** screen to get these details from the user. In the current task, you use three strings as input to the ViewModel, convert them to an `Item` entity instance, and save it to the database using the `ItemDao` instance. It's time to implement.

1. In the `InventoryViewModel` class, add a private function called `insertItem()` that takes in an `Item` object and adds the data to the database in a non-blocking way.

```
private fun insertItem(item: Item) {  
}
```

2. To interact with the database off the main thread, start a coroutine and call the DAO method within it. Inside the `insertItem()` method, use the `viewModelScope.launch` to start a coroutine in the `ViewModelScope`. Inside the launch function, call the suspend function `insert()` on `itemDao` passing in the `item`. The `ViewModelScope` is an extension property to the `ViewModel` class that automatically cancels its child coroutines when the `ViewModel` is destroyed.

```
private fun insertItem(item: Item) {  
    viewModelScope.launch {  
        itemDao.insert(item)  
    }  
}
```

Import `kotlinx.coroutines.launch`, `androidx.lifecycle.viewModelScope`

`com.example.inventory.data.Item`, if not automatically imported.

Note: Throughout the codelab import `com.example.inventory.data.Item` for `Item` entity, when requested by Android Studio.

3. In the `InventoryViewModel` class, add another private function that takes in three strings and returns an `Item` instance.

```
private fun getNewItemEntry(itemName: String, itemPrice: String, itemCount:  
String): Item {  
    return Item(  
        itemName = itemName,  
        itemPrice = itemPrice.toDouble(),  
        quantityInStock = itemCount.toInt()  
    )  
}
```

4. Still inside the `InventoryViewModel` class, add a public function called `addNewItem()` that takes in three strings for item details. Pass in item detail strings to `getNewItemEntry()` function and assign the returned value to a val named `newItem`. Make a call to `insertItem()` passing in the `newItem` to add the new entity to the database. This will be called from the UI fragment to add `Item` details to the database.

```
fun addNewItem(itemName: String, itemPrice: String, itemCount: String) {
    val newItem = getNewItemEntry(itemName, itemPrice, itemCount)
    insertItem(newItem)
}
```

Notice that you did not use `viewModelScope.launch` for `addNewItem()`, but it is needed above in `insertItem()` when you call a DAO method. The reason is that the *suspend functions are only allowed to be called from a coroutine or another suspend function*. The function `itemDao.insert(item)` is a suspend function.

You have added all the required functions to add entities to the database. In the next task you will update the **Add Item** fragment to use the above functions.

9. Update AddItemFragment

1. In `AddItemFragment.kt`, at the beginning of the `AddItemFragment` class create a private val called `viewModel` of the type `InventoryViewModel`. Use the `by activityViewModels()` Kotlin property delegate to share the `ViewModel` across fragments. You will fix the error in the next step.

```
private val viewModel: InventoryViewModel by activityViewModels {
}
```

2. Inside the lambda, call the `InventoryViewModelFactory()` constructor and pass in the `ItemDao` instance. Use the `database` instance you created in one of the previous tasks to call the `itemDao` constructor.

```
private val viewModel: InventoryViewModel by activityViewModels {
    InventoryViewModelFactory(
        (activity?.application as InventoryApplication).database
            .itemDao()
    )
}
```

Tip: This is mostly boilerplate code, so you can reuse the code for future to create a `ViewModel` instance using a `ViewModel` factory.

3. Below the `viewModel` definition, create a `lateinit var` called `item` of the type `Item`.

```
lateinit var item: Item
```

4. The **Add Item** screen contains three text fields to get the item details from the user. In this step, you will add a function to verify if the text in the `TextFields` are not empty. You will use this function to verify user input before adding or updating the entity in the database. This validation needs to be done in the `ViewModel` and not in the `Fragment`. In

the `InventoryViewModel` class, add the following public function called `isEntryValid()`.

```
fun isEntryValid(itemName: String, itemPrice: String, itemCount: String): Boolean {
    if (itemName.isBlank() || itemPrice.isBlank() || itemCount.isBlank()) {
        return false
    }
    return true
}
```

5. In `AddItemFragment.kt`, below the `onCreateView()` function create a private function called `isEntryValid()` that returns a `Boolean`. You will fix the missing return value error in the next step.

```
private fun isEntryValid(): Boolean {
}
```

6. In the `AddItemFragment` class, implement the `isEntryValid()` function. Call the `isEntryValid()` function on the `viewModel` instance, passing in the text from the text views. Return the value of the `viewModel.isEntryValid()` function.

```
private fun isEntryValid(): Boolean {
    return viewModel.isEntryValid(
        binding.itemName.text.toString(),
        binding.itemPrice.text.toString(),
        binding.itemCount.text.toString()
    )
}
```

7. In the `AddItemFragment` class below the `isEntryValid()` function, add another private function called `addNewItem()` with no parameters and return nothing. Inside the function, call `isEntryValid()` inside the `if` condition.

```
private fun addNewItem() {
    if (isEntryValid()) {
    }
}
```

8. Inside the `if` block, call the `addNewItem()` method on the `viewModel` instance. Pass in the item details entered by the user, use the `binding` instance to read them.

```
if (isEntryValid()) {
    viewModel.addNewItem(
        binding.itemName.text.toString(),
        binding.itemPrice.text.toString(),
        binding.itemCount.text.toString(),
    )
}
```

9. Below the `if` block, create a `val` action to navigate back to the `ItemListFragment`. Call `findNavController().navigate()`, passing in the action.

```
val action =
AddItemFragmentDirections.actionAddItemFragmentToItemListFragment()
findNavController().navigate(action)
```

Import `androidx.navigation.fragment.findNavController`.

10. The complete method should look like the following.

```
private fun addNewItem() {
    if (isEntryValid()) {
        viewModel.addNewItem(
            binding.itemName.text.toString(),
            binding.itemPrice.text.toString(),
            binding.itemCount.text.toString(),
        )
        val action =
AddItemFragmentDirections.actionAddItemFragmentToItemListFragment()
        findNavController().navigate(action)
    }
}
```

11. To tie everything together, add a click handler to the **Save** button. In the `AddItemFragment` class, above the `onDestroyView()` function, override the `onViewCreated()` function.

12. Inside the `onViewCreated()` function, add a click handler to the save button, and call `addNewItem()` from it.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    binding.saveAction.setOnClickListener {
        addNewItem()
    }
}
```

13. Build and run your app. Tap the + Fab. In the **Add Item** screen, add the item details and tap **Save**. This action saves the data, but you cannot see anything yet in the app. In the next task, you will use the [Database Inspector](#) to view the data you saved.



Add Item

Item Name *

Apples

Item Price *

\$ 2.00

Quantity in Stock *

100

SAVE

1

2

3

-

4

5

6

⌋

7

8

9

⌫

,

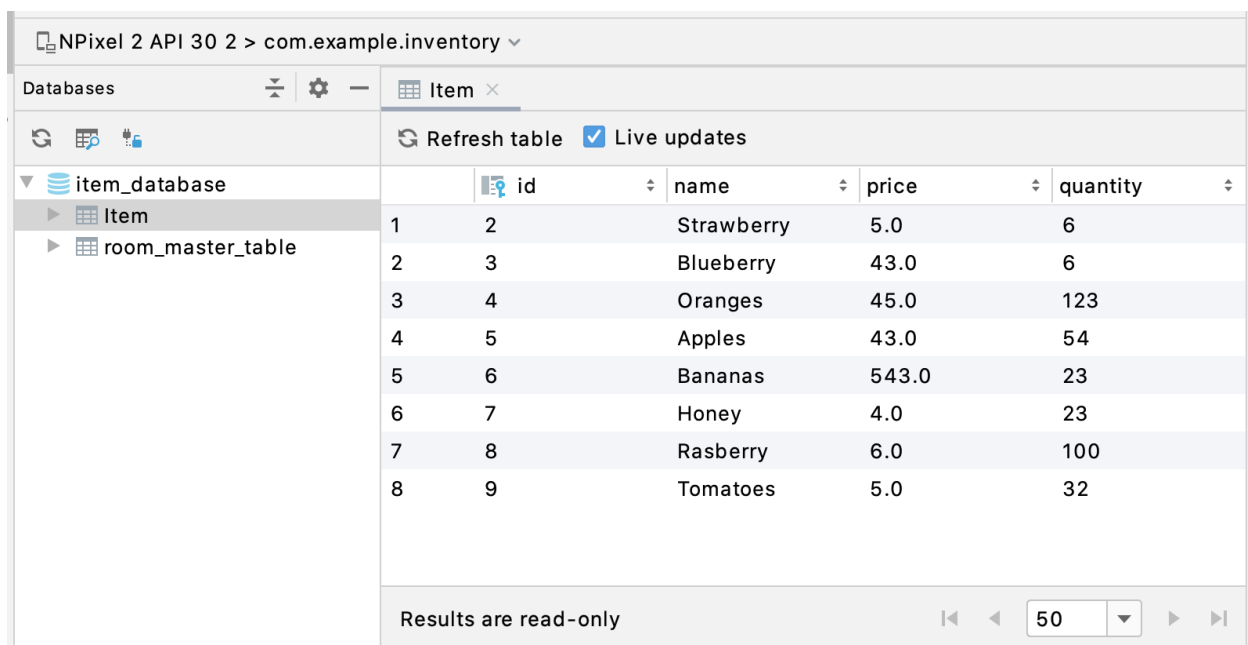
0

.

✓

View the database using Database Inspector

1. Run your app on an emulator or connected device running API level 26 or higher, if you have not done so already. [Database Inspector](#) works best on emulator/devices running API level 26.
2. In Android studio, select **View > Tool Windows > Database Inspector** from the menu bar.
3. In the Database Inspector pane, select the `com.example.inventory` from the dropdown menu.
4. The **item_database** in the Inventory app appears in the **Databases** pane. Expand the node for the **item_database** and select **Item** to inspect. If your **Databases** pane is empty, use your emulator to add some items to the database using the **Add Item** screen.
5. Check the **Live updates** checkbox in the Database Inspector to automatically update the data it presents as you interact with your running app in the emulator or device.



NPixel 2 API 30 2 > com.example.inventory ▾				
Databases		Item ×		
Refresh table		☒ Live updates		
	id	name	price	quantity
1	2	Strawberry	5.0	6
2	3	Blueberry	43.0	6
3	4	Oranges	45.0	123
4	5	Apples	43.0	54
5	6	Bananas	543.0	23
6	7	Honey	4.0	23
7	8	Raspberry	6.0	100
8	9	Tomatoes	5.0	32

Results are read-only

Congratulations! You have created an app that can persist the data using [Room](#). In the next codelab, you will add a `RecyclerView` to your app to display the items on the database and add new features to the app like deleting and updating the entities. See you there!

[10. Solution code](#)

The solution code for this codelab is in the GitHub repo and branch shown below.

Solution Code URL:

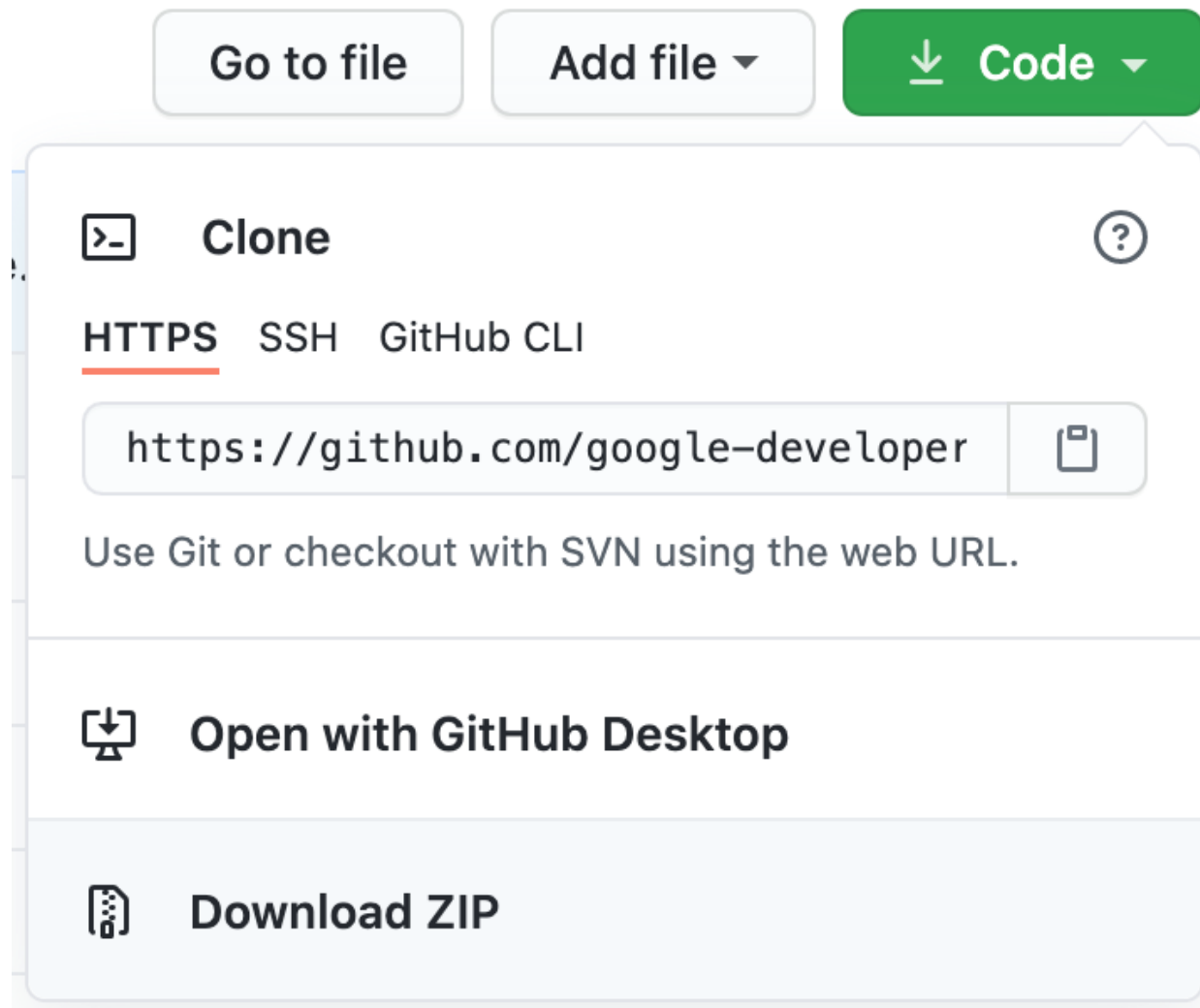
<https://github.com/google-developer-training/android-basics-kotlin-inventory-app/tree/room>

Branch: room

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

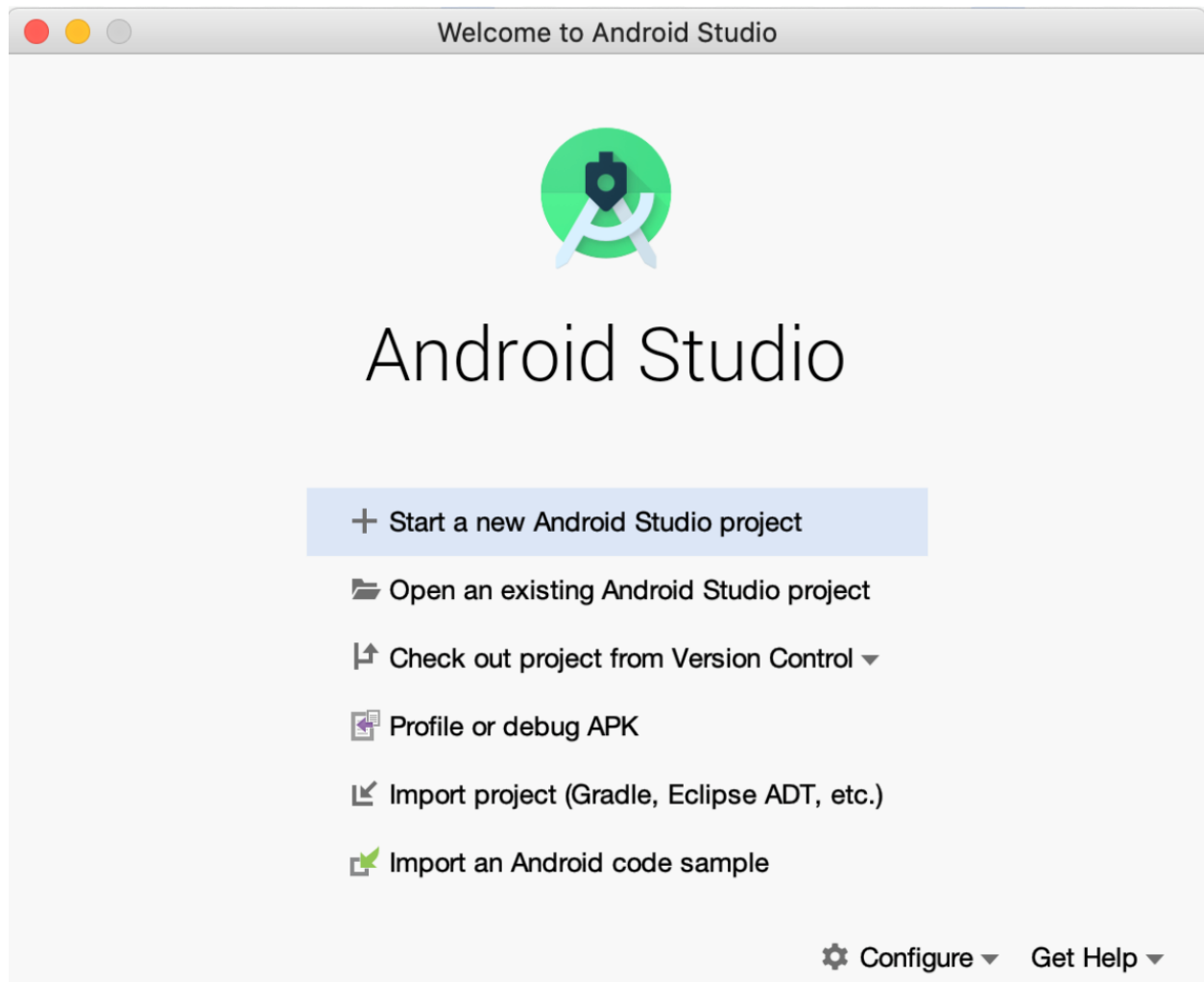
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



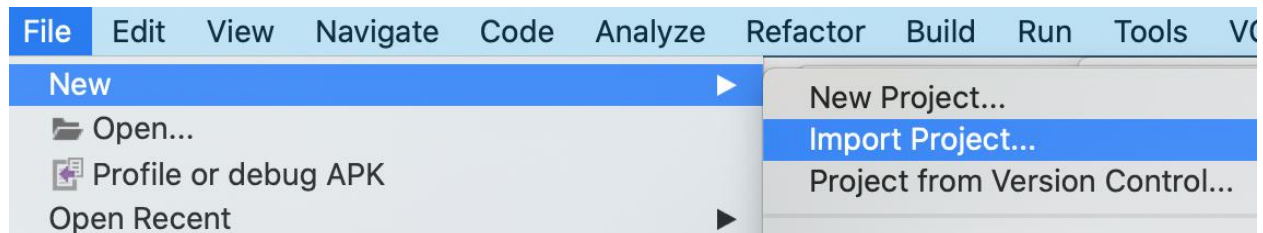
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.
7. Browse the project files in the **Project** tool window to see how the app is set-up.

11. Summary

- Define your tables as data classes annotated with `@Entity`. Define properties annotated with `@ColumnInfo` as columns in the tables.
- Define a data access object (DAO) as an interface annotated with `@Dao`. The DAO maps Kotlin functions to database queries.
- Use annotations to define `@Insert`, `@Delete`, and `@Update` functions.
- Use the `@Query` annotation with an SQLite query string as a parameter for any other queries.
- Use [Database Inspector](#) to view the data saved in the Android SQLite database.

12. Learn more

Android Developer Documentation

- [Save data in a local database using Room](#)
- [androidx.room](#)
- [Debug your database with the Database Inspector](#)

Blog posts

- [7 Pro-tips for Room](#)
- [The one and only object. Kotlin Vocabulary](#)

Videos

- [Kotlin: Using Room Kotlin APIs](#)
- [Database Inspector](#)

Other documentation and articles

- [Singleton pattern](#)
- [Companion objects](#)
- [SQLite Tutorial - An Easy Way to Master SQLite Fast](#)

